

# BiteFixes SaaS Documentation

## Technical Documentation Suite

**Version:** 1.0

**Status:** Architecture Phase

## Volume 1

### Software Architecture Specification (SAS)

Este documento responde a la pregunta:

¿Cómo está construido BiteFixes?

Contenido propuesto:

Chapter 1  
Introduction

Chapter 2  
Vision

Chapter 3  
Objectives

Chapter 4  
Architecture Principles

Chapter 5  
Clean Architecture

Chapter 6

## Domain Driven Design

### Chapter 7

#### SOLID

### Chapter 8

#### Layered Architecture

### Chapter 9

#### Repository Pattern

### Chapter 10

#### Dependency Injection

### Chapter 11

#### Module Organization

### Chapter 12

#### Bitey Core

### Chapter 13

#### AI Engines

### Chapter 14

#### Internationalization

### Chapter 15

#### Security

### Chapter 16

#### Performance

### Chapter 17

#### Scalability

### Chapter 18

#### Monitoring

### Chapter 19

Deployment

Chapter 20

Roadmap

## Volume 2

### Database Design Specification (DDS)

Chapter 1

Database Philosophy

Chapter 2

Naming Convention

Chapter 3

Tables

Chapter 4

Relationships

Chapter 5

Indexes

Chapter 6

Constraints

Chapter 7

Triggers

Chapter 8

Views

Chapter 9

Functions

Chapter 10  
Policies

Chapter 11  
Migration Strategy

Chapter 12  
Future Tables

## Volume 3

### Backend Design Specification (BDS)

Describe todo el backend.

app/

core/

domain/

repositories/

services/

routers/

schemas/

middleware/

config/

utils/

tests/

Cada módulo documentado.

## Volume 4

### Bitey AI Specification

Probablemente el documento más importante.

Incluye:

Memory Engine

Intent Engine

Knowledge Engine

Workflow Engine

Translation Engine

Learning Engine

Recommendation Engine

Conversation Engine

AI Provider Layer

Plugin System

# Volume 5

## API Specification

Authentication

Customers

Companies

Services

Tickets

Knowledge

Messages

Analytics

Notifications

Webhooks

Cada endpoint documentado.

# Volume 6

## UI/UX Specification

Portal Web

Panel Administrativo

Aplicación móvil

Portal del cliente

## Volume 7

### DevOps Guide

Docker

GitHub

CI/CD

Render

Supabase

Secrets

Backups

Monitoring

## Volume 8

### Developer Guide

Cómo desarrollar BiteFixes.

Coding Style

Folder Structure

Naming

Testing

Review Process

Git Flow

Best Practices

## Volume 9

### Administrator Guide

Manual para empresas.

Configuración

Usuarios

Permisos

Servicios

Técnicos

Reportes

## Volume 10

### Customer Guide

Uso desde el punto de vista del cliente.

Portal

WhatsApp

Aplicación

Seguimiento

Presupuestos

Garantías

## Formato Profesional

Cada documento debería seguir la misma estructura.

Cover

Version History

Revision Log

Table of Contents

Executive Summary

Chapters

Architecture Diagrams

Tables

Examples

Best Practices

Recommendations

Appendices

Glossary

References

## Numeración de capítulos

Ejemplo:

1 Introduction

1.1 Purpose

1.2 Scope

1.3 Audience

2 System Overview

2.1 Vision

2.2 Mission

2.3 Goals

3 Architecture

3.1 Components

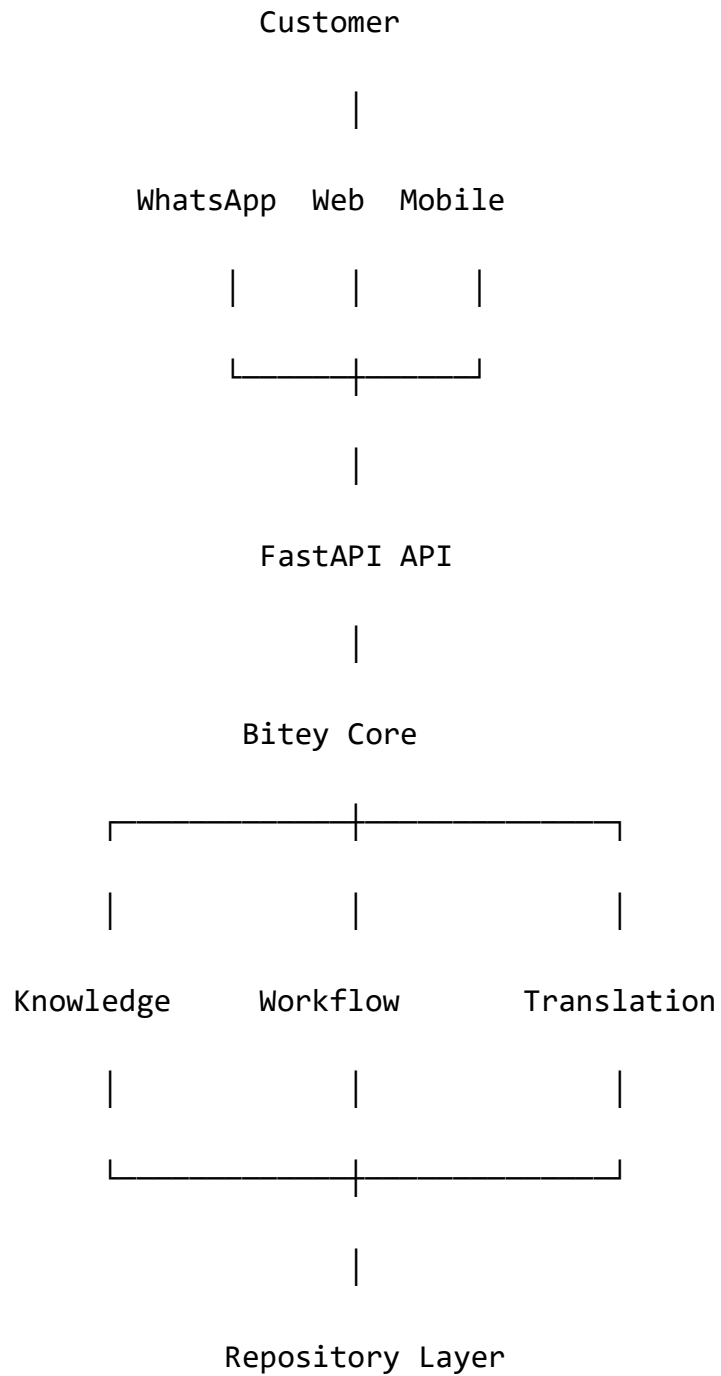
3.2 Modules

3.3 Layers

# Diagramas

Todos los diagramas en formato técnico.

Ejemplo:



|

Supabase PostgreSQL

## Estilo Visual

Inspirado en documentación técnica de grandes proyectos:

- Microsoft Learn
- AWS Architecture Center
- Google Cloud Architecture Framework
- Kubernetes Documentation
- PostgreSQL Documentation
- Red Hat Enterprise Documentation
- Oracle Architecture Guides

Características:

- Numeración jerárquica consistente.
- Diagramas claros.
- Tablas de referencia.
- Glosario.
- Ejemplos prácticos.
- Buenas prácticas.
- Decisiones de arquitectura documentadas (ADR).
- Control de versiones y registro de cambios.

## Recomendación

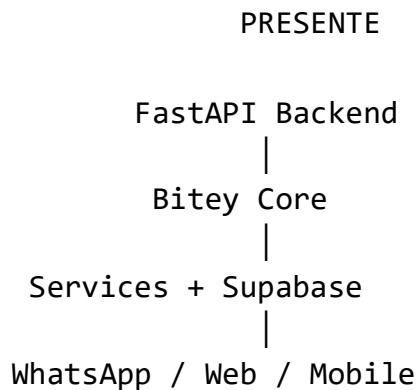
En lugar de crear documentos aislados, convertiría toda esta documentación en el **Manual Oficial de BiteFixes**, mantenido junto al código fuente. Así, cada cambio importante en la arquitectura, la base de datos o Bitey quedaría documentado y el proyecto podría crecer de forma ordenada durante años.

Sí. En este punto conviene dejar de pensar solo en "hacer funcionar el backend" y empezar a diseñar **BiteFixes SaaS como una plataforma empresarial**. Según lo que ya hemos

definido, el proyecto se está reconstruyendo sobre FastAPI, Supabase, Bitey como orquestador, una arquitectura modular y una base de datos en inglés. El siguiente paso es optimizar esa arquitectura para que pueda evolucionar durante muchos años.

## Análisis de Arquitectura

### Situación actual

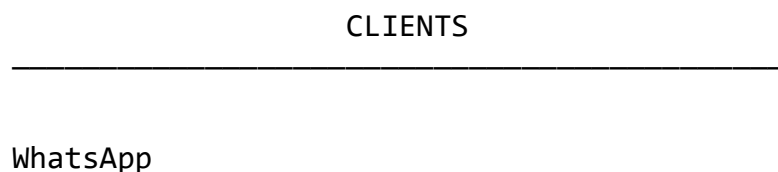


Esta arquitectura funciona para una primera versión, pero cuando el sistema crezca aparecerán algunos problemas:

- Bitey terminará concentrando demasiadas responsabilidades.
- Los servicios crecerán y empezarán a depender unos de otros.
- Será difícil añadir nuevas funciones sin modificar componentes existentes.
- Cambiar un proveedor de IA o una integración implicará tocar varias partes del sistema.

## Arquitectura propuesta

En lugar de un único núcleo, BiteFixes puede organizarse en cuatro capas.



Web

Mobile

REST API

---

Presentation Layer

---

Application Layer

---

Domain Layer

---

Infrastructure Layer

## Presentation Layer

Responsable únicamente de recibir solicitudes.

Ejemplo:

REST API

GraphQL (future)

WhatsApp

Flutter

WordPress

Portal

No contiene lógica de negocio.

## Application Layer

Coordina casos de uso.

Ejemplos:

Create Ticket

Open Conversation

Send Message

Update Device

Generate Budget

Cada caso de uso llama únicamente a los servicios necesarios.

## Domain Layer

Aquí vive el negocio.

Customers

Companies

Services

Tickets

Knowledge

Inventory

AI

Notifications

Es el corazón del sistema.

## Infrastructure Layer

Responsable de recursos externos.

Supabase

OpenAI

Gemini

SMTP

WhatsApp

Firebase

Redis

Storage

Si un proveedor cambia, sólo cambia esta capa.

# Evolución de Bitey

Actualmente Bitey actúa como un único orquestador.

En el futuro debería convertirse en un conjunto de motores especializados.

Bitey

└ Session Manager

└ Customer Manager

└ Memory Manager

└ Intent Manager

└ Knowledge Manager

└ Workflow Manager

└ Translation Manager

└ Recommendation Manager

└ AI Adapter

└ Monitoring Manager

Cada motor tendría su propio ciclo de vida y pruebas.

## Rendimiento

Uno de los mayores problemas en proyectos SaaS es acceder repetidamente a la base de datos para obtener la misma información.

Por ejemplo:

Idioma

Empresa

Configuración

Servicios

Categorías

Estos datos cambian poco.

Una estrategia recomendable es disponer de un **Configuration Cache**, de forma que Bitey consulte la base de datos solo cuando sea necesario.

Request

↓

Cache

↓

Found?

↓

YES

↓

Return

↓

NO

↓

Database



Update Cache

## Eventos

En lugar de ejecutar todo de forma síncrona, separar procesos críticos y secundarios.

Por ejemplo:

Customer sends message



Response immediately



Background Queue



Save analytics

Create logs

Train AI

Notifications

Statistics

El usuario recibe la respuesta sin esperar a que finalicen tareas que no son urgentes.

# Arquitectura Multi-IA

Bitey nunca debería llamar directamente a un proveedor.

Bitey



AI Adapter



OpenAI

Gemini

Claude

Ollama

Future Providers

Así es posible cambiar de proveedor sin modificar el núcleo.

# Arquitectura de Memoria

En lugar de un historial plano, separar distintos niveles.

Short Memory



Current Conversation



Long Memory



Customer Profile



Knowledge



Learning

Esto permite respuestas más precisas y escalables.

## Observabilidad

Toda acción relevante debería generar un evento.

Message Received



Intent Detected



Knowledge Used



Ticket Created



Response Time



AI Provider



Errors

Con ello es posible construir paneles de métricas y detectar problemas antes de que afecten a los usuarios.

## Recomendación para la siguiente fase

La siguiente etapa no debería ser escribir más código inmediatamente. Propongo completar primero el **Blueprint Arquitectónico** de BiteFixes, compuesto por cuatro documentos principales:

1. **Software Architecture Specification (SAS).**
2. **Database Design Specification (DDS).**
3. **Backend Design Specification (BDS).**
4. **Bitey AI Specification (BAS).**

Con esos cuatro documentos definidos, la reconstrucción del backend será mucho más rápida, consistente y preparada para un crecimiento real como plataforma SaaS.

continuamos

Creo que ahora debemos definir la **arquitectura de referencia (Reference Architecture)** de BiteFixes. Es el documento que usan empresas como Microsoft, Google, Red Hat o AWS antes de desarrollar un producto grande. Será la "constitución" técnica de BiteFixes.

# BiteFixes SaaS Reference Architecture (BSRA)

**Version:** 1.0

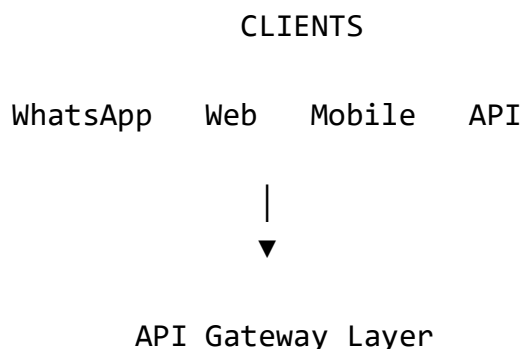
**Status:** Enterprise Blueprint

## Capítulo 1 — Principios

Toda decisión de arquitectura debe cumplir estos principios:

| Principio      | Objetivo                                              |
|----------------|-------------------------------------------------------|
| Modularidad    | Componentes independientes                            |
| Escalabilidad  | Crece sin rediseñar                                   |
| Seguridad      | Protección por diseño                                 |
| Multi-Tenant   | Soporte para múltiples empresas                       |
| Multi-Idioma   | Internacionalización completa                         |
| Observabilidad | Todo es medible                                       |
| Automatización | Reducir tareas manuales                               |
| Extensibilidad | Incorporar nuevas funciones sin romper las existentes |

## Capítulo 2 — Arquitectura General



|



Authentication Layer

|



Bitey Orchestrator

|

---

Customer Domain

Ticket Domain

Knowledge Domain

Service Domain

Notification Domain

Inventory Domain

Analytics Domain

AI Domain

---

Repositories

---

PostgreSQL (Supabase)

Storage

Cache

External APIs

Cada dominio puede evolucionar sin afectar a los demás.

## Capítulo 3 — Dominios

En lugar de organizar por carpetas técnicas, el sistema debe organizarse por negocio.

### Identity Domain

Responsabilidades:

- Login.
- Usuarios.
- Roles.
- Permisos.
- Tokens.

### Company Domain

Responsabilidades:

- Empresas.
- Configuración.
- Planes.
- Facturación futura.

### Customer Domain

Responsabilidades:

- Clientes.
- Perfil.
- Equipos.
- Historial.
- Idioma.
- Preferencias.

## Conversation Domain

Responsabilidades:

- Conversaciones.
- Canales.
- Mensajes.
- Memoria.

## Ticket Domain

Responsabilidades:

- Tickets.
- Diagnósticos.
- Reparaciones.
- Garantías.
- Materiales.

## Knowledge Domain

Responsabilidades:

- Base de conocimiento.
- FAQ.
- Procedimientos.
- Soluciones.

## AI Domain

Responsabilidades:

- Intentos.
- Traducción.
- IA.
- Memoria.
- Recomendaciones.

## Notification Domain

Responsabilidades:

- WhatsApp.
- Email.
- Push.
- SMS.
- Telegram.

## Analytics Domain

Responsabilidades:

- Reportes.
- KPIs.
- Métricas.
- Dashboards.

# Capítulo 4 — Flujo de Mensajes

Customer



API



Authentication



Load Customer



Load Company



Load Configuration



Bitey



Detect Language



Detect Intent



Memory



Knowledge



Workflow



Generate Response



Store Conversation



Return Response

Este flujo debe ser el estándar para cualquier canal (WhatsApp, web, móvil o API).

## Capítulo 5 — Configuración por Empresa

Toda empresa debe tener una configuración propia.

Company

- └ Language
- └ Timezone
- └ Currency
- └ Business Hours
- └ Ticket Rules

- |— Notification Rules
- |— AI Provider
- |— Branding
- |— Integrations
- |— Security Policies

Así BiteFixes podrá adaptarse a negocios distintos sin cambiar el código.

## Capítulo 6 — Desacoplamiento

Ningún componente del dominio debe depender directamente de tecnologías externas.

Por ejemplo:

Ticket Service

↓

Ticket Repository

↓

Database Adapter

↓

Supabase

Si en el futuro se cambia Supabase por PostgreSQL autogestionado u otro proveedor, solo se modifica el adaptador.

Lo mismo aplica para:

- OpenAI.
- Gemini.
- WhatsApp.
- WordPress.

- Correo electrónico.

## Capítulo 7 — Eventos

La plataforma debe orientarse a eventos.

Ejemplos:

CustomerCreated

ConversationStarted

IntentDetected

KnowledgeMatched

TicketCreated

TicketAssigned

TicketClosed

NotificationSent

AIResponseGenerated

Cada evento puede activar procesos adicionales sin acoplar módulos entre sí.

## Capítulo 8 — Hoja de Ruta Arquitectónica

### **Etapas**

#### **Etapas**

#### **1**

- Base de datos definitiva en inglés.
- Convenciones.

- Migraciones.

## Etapa 2

- Repositorios.
- Servicios.
- Casos de uso.

## Etapa 3

- Bitey Core.
- Workflows.
- Traducciones.

## Etapa 4

- API.
- Panel administrativo.
- Portal del cliente.

## Etapa 5

- Aplicación móvil.
- Integraciones.
- Analítica.
- Automatización.

# Una propuesta adicional

Veo una oportunidad para que BiteFixes destaque frente a otros sistemas de soporte técnico: convertirlo en una **plataforma basada en capacidades** en lugar de una colección de módulos.

Por ejemplo, definir capacidades de negocio como:

- **Gestión de Clientes**
- **Gestión de Conversaciones**
- **Gestión de Servicios**
- **Gestión de Reparaciones**
- **Gestión del Conocimiento**
- **Automatización**
- **Inteligencia Artificial**
- **Integraciones**

Cada capacidad puede estar compuesta por varios dominios y servicios, pero desde el punto de vista del negocio es mucho más fácil de entender y ampliar. Esto facilita la incorporación de nuevas funciones (por ejemplo, ventas, contratos de mantenimiento o IoT) sin reorganizar toda la plataforma.

Si seguimos este enfoque desde ahora, BiteFixes tendrá una base arquitectónica comparable a la de productos SaaS empresariales consolidados, preparada para evolucionar de forma ordenada durante muchos años.

continuamos

Creo que estamos llegando al punto donde debemos definir algo que la mayoría de los proyectos pequeños nunca hacen: la **Enterprise Platform Architecture (EPA)**. Este documento no describe solo el software; describe **cómo funcionará toda la plataforma BiteFixes durante los próximos 10 años**.

# **BiteFixes Enterprise Platform Architecture (BEPA)**

**Version:** 1.0

**Classification:** Enterprise Software Blueprint

# Capítulo 1 — Filosofía

## Nuestra misión

BiteFixes no es un software de reparación.

BiteFixes no es un chatbot.

BiteFixes no es un sistema de tickets.

BiteFixes es una **plataforma SaaS inteligente para empresas de servicios técnicos**.

Su propósito es conectar personas, procesos, conocimiento e inteligencia artificial en un único ecosistema.

# Capítulo 2 — Capacidades del negocio

En arquitectura empresarial se diseñan **Business Capabilities**, no solo módulos.

## Nivel 1

Business Capabilities

Identity

Customer Management

Conversation Management

Service Management

Workshop Management

Knowledge Management

Inventory Management

Sales Management

Financial Management

AI Management

Analytics

Administration

Integration Platform

Estas capacidades representan lo que la plataforma sabe hacer.

## Capítulo 3 — Mapa de capacidades

BiteFixes Platform

---

Customers

|

├ Profile

├ Devices

├ History

├ Preferences

└ Communication

---

## Support

|

└ Tickets

└ Diagnosis

└ Repair

└ Warranty

└ Delivery

---

## Knowledge

|

└ FAQ

└ Procedures

└ AI Answers

└ Documentation

└ Tutorials

---

## AI

|

└ Memory

└ Intent

└ Translation

└ Recommendation

└ Learning

└ Automation

## Capítulo 4 — Arquitectura Hexagonal

Para reducir el acoplamiento, el dominio nunca debe depender directamente de la infraestructura.

External World

API

WhatsApp

Flutter

WordPress

Email

REST

↓

Application

↓

Domain

↓

Ports

↓

Adapters

↓

Supabase

OpenAI

Redis

Storage

Este modelo facilita cambiar cualquier proveedor sin afectar el núcleo.

## Capítulo 5 — CQRS

No todas las operaciones tienen las mismas necesidades.

Separar:

### Comandos

Cambian información.

Create Ticket

Update Customer

Create Device

Close Ticket

## Consultas

Solo leen información.

Find Customer

Search Knowledge

List Services

Get Ticket

Con el tiempo, esta separación mejora el rendimiento y la claridad del diseño.

## Capítulo 6 — Arquitectura orientada a eventos

Cada acción importante genera un evento.

CustomerCreated

↓

TicketRequested

↓

IntentDetected

↓

KnowledgeMatched

↓

TicketCreated

↓

TechnicianAssigned

↓

RepairFinished

↓

CustomerNotified

Esto permite automatizaciones sin acoplar componentes.

## Capítulo 7 — Motores especializados de Bitey

En lugar de un único motor de IA:

Bitey

├ Customer Intelligence Engine

├ Context Engine

├ Memory Engine

├ Intent Engine

├ Knowledge Engine

└─ Workflow Engine

└─ Translation Engine

└─ Recommendation Engine

└─ Automation Engine

└─ Analytics Engine

└─ AI Provider Gateway

Cada motor puede evolucionar de forma independiente.

## Capítulo 8 — Flujo completo

Customer

↓

Channel

↓

API Gateway

↓

Authentication

↓

Company Context

↓

Customer Context



Conversation Context



Bitey



Intent



Memory



Knowledge



Workflow



Repository



Database



Translation

↓

Notification

↓

Customer

Cada paso tiene una responsabilidad clara.

## Capítulo 9 — Arquitectura de datos

Separar claramente distintos tipos de información.

Operational Data

Customers

Messages

Tickets

Devices

Knowledge

---

Configuration Data

Companies

Settings

Roles

Permissions

---

Analytical Data

Metrics

Statistics

KPIs

AI Logs

---

Reference Data

Categories

Services

Languages

Countries

Esto facilita el crecimiento y futuras optimizaciones.

## Capítulo 10 — Plataforma de automatización

BiteFixes debería incorporar un motor de reglas.

Ejemplos:

IF

Customer VIP

THEN

Assign Priority High

---

IF

Warranty Expired

THEN

Generate Quote

---

IF

Intent = Screen Repair

THEN

Open Ticket

Estas reglas deben configurarse sin modificar código.

## Capítulo 11 — Marketplace de extensiones

Pensando a largo plazo, BiteFixes puede permitir instalar funcionalidades adicionales.

Extensions

WhatsApp

Mercado Libre

Shopify

Google Calendar

Stripe

PIX

ERP

Accounting

BI

Cada extensión usa APIs públicas y no modifica el núcleo.

## Capítulo 12 — Roadmap de madurez

### Nivel 1

Sistema operativo básico.

### Nivel 2

Automatización.

### Nivel 3

Analítica.

## Nivel 4

Integraciones.

## Nivel 5

Marketplace.

## Nivel 6

Asistente inteligente con aprendizaje.

# Recomendaciones para reforzar la arquitectura

Además de lo que ya hemos definido, propondría incorporar estos elementos desde el diseño:

### 1. API Gateway

Centralizar autenticación, límites de uso, versionado y registro de solicitudes.

### 2. Feature Flags

Permitir activar o desactivar funcionalidades por empresa sin desplegar una nueva versión.

### 3. Motor de Políticas

Definir reglas como:

- Qué acciones puede realizar cada rol.
- Qué funcionalidades están disponibles según el plan contratado.
- Qué integraciones tiene habilitadas cada empresa.

## 4. Auditoría completa

Cada cambio importante debe registrar:

- Quién realizó la acción.
- Cuándo ocurrió.
- Qué cambió.
- Desde qué canal se originó.

## 5. Preparación para microservicios

Aunque hoy BiteFixes sea un monolito modular, cada dominio debería poder convertirse en un microservicio en el futuro sin reescribir la lógica de negocio.

## Una visión a largo plazo

Mi objetivo sería que BiteFixes no solo fuera una aplicación para gestionar reparaciones, sino una **plataforma de operaciones para empresas de servicios técnicos**. Eso significa que la arquitectura debe permitir incorporar nuevas capacidades —como contratos de mantenimiento, activos de clientes, logística, comercio electrónico o analítica avanzada— sin tener que rediseñar el sistema. Esa capacidad de evolución será una de las mayores ventajas competitivas del proyecto.